Getting Random Values in C and C++ with Rand

Written by RoD

At some point in any programmer's life, he or she must learn how to get a random value, or values, in their program. To some this seems involved, difficult, or even beyond their personal ability. This, however, is simply not the case.

Randomizing of values is, at its most basic form, one of the easier things a programmer can do with the C++ language. I have created this short tutorial for Cprogramming.com to aid you in learning, constructing, and using the functions available to you to randomize values.

I will first start with an introduction to the idea of randomizing values, followed by a simple example program that will output three random values. Once a secure understanding of these concepts is in place (hopefully it will be), I will include a short program that uses a range of values from which the random values can be taken.

Ok, now that you know why this tutorial was written, and what it includes, you are ready to learn how to randomize values! So without further ado, lets get started, shall we?

Many programs that you will write require the use of random numbers. For example, a game such as backgammon requires a roll of two dice on each move. Since there are 6 numbers on each die, you could calculate each roll by finding a random number from 1 to 6 for each die.

To make this task a little easier, C++ provides us with a library function, called rand that returns an integer between 0 and RAND_MAX. LetÕs take a break to explain what RAND_MAX is. RAND_MAX is a compiler-dependent constant, and it is inclusive. Inclusive means that the value of RAND_MAX is included in the range of values. The function, rand, and the constant, RAND_MAX, are included in the library header file stdlib.h.

The number returned by function rand is dependent on the initial value, called a seed that remains the same for

each run of a program. This means that the sequence of random numbers that is generated by the program will be exactly the same on each run of the program.

How do you solve this problem you ask? Well I'll tell you! To help us combat this problem we will use another function, srand(seed), which is also declared in the stdlib.h header file. This function allows an application to specify the initial value used by rand at program startup.

Using this method of randomization, the program will use a different seed value on every run, causing a different set of random values every run, which is what we want in this case. The problem posed to us now, of course, is how to get an arbitrary seed value. Forcing the user or programmer to enter this value every time the program was run wouldnÕt be very efficient at all, so we need another way to do it.

So we turn to the perfect source for our always-changing value, the system clock. The C++ data type time_t and the function time, both declared in time.h, can be used to easily retrieve the time on the computers clock.

When converted to an unsigned integer, a positive whole number, the program time (at execution of program) can make a very nice seed value. This works nicely because no two program executions will occur at the same instant of the computers clock.

As promised, here is a very basic example program. The following code was written in Visual C++ 6.0, but should compile fine on most computers (given u have a compiler, which if your reading this I assume you do). The program outputs three random values.

```
/*Steven Billington
January 17, 2003
Ranexample.cpp
Program displays three random integers.
*/
/*
Header: iostream
Reason: Input/Output stream
Header: cstdlib
Reason: For functions rand and srand
Header: time.h
Reason: For function time, and for data type time_t
*/
#include <iostream>
#include <cstdlib>
#include <time.h>
int main()
{
/*
Declare variable to hold seconds on clock.
*/
time_t seconds;
/*
Get value from system clock and
place in seconds variable.
*/
time(&seconds);
/*
Convert seconds to a unsigned
integer.
*/
srand((unsigned int) seconds);
/*
Output random values.
*/
cout<< rand() << endl;
cout<< rand() << endl;
cout<< rand() << endl;
return 0;
}
```

Users of a random number generator might wish to have a narrower or a wider range of numbers than provided by the rand function. Ideally, to solve this problem a user would specify the range with integer values representing the lower and the upper bounds. To understand how we might accomplish this with the rand function, consider how to generate a number between 0 and an arbitrary upper bound, referrred to as high, inclusive.

For any two integers, say a and b, a % b is between 0 and b - 1, inclusive. With this in mind, the expression rand () % high + 1 would generate a number between 1 and high, inclusive, where high is less than or equal to RAND_MAX, a constant defined by the compiler. To place a lower bound in replacement of 1 on that result, we can have the program generate a random number between 0 and (high - low + 1) + low.

I realize how confused you might be right now, so take a look at the next sample program I promised, run it, toy with it, and alternate it to give you different values. It has been a pleasure to teach you another chapter in the world of C++, and you may feel free to email me at Silent_Death17@hotmail.com or to contact me on the message boards of this fine website, where I use the name RoD.

Enjoy, and happy programming!

```
/*
Steven Billington
January 17, 2003
exDice.cpp
Program rolls two dice with random
results.
*/
/*
Header: iostream
Reason: Input/Output stream
Header: stdlib
Reason: For functions rand and srand
Header: time.h
Reason: For function time, and for data type time_t
*/
#include <iostream>
#include <cstdlib>
#include <time.h>
/*
These constants define our upper
and our lower bounds. The random numbers
will always be between 1 and 6, inclusive.
*/
const int LOW = 1;
const int HIGH = 6;
int main()
{
/*
Variables to hold random values
for the first and the second die on
each roll.
*/
int first_die, sec_die;
/*
Declare variable to hold seconds on clock.
*/
time_t seconds;
/*
Get value from system clock and
place in seconds variable.
*/
time(&seconds);
/*
Convert seconds to a unsigned
integer.
*/
srand((unsigned int) seconds);
```

```
srand((unsigned int) seconds);
/*
Get first and second random numbers.
*/
first_die = rand() % (HIGH - LOW + 1) + LOW;
sec_die = rand() % (HIGH - LOW + 1) + LOW;
/*
Output first roll results.
*/
cout<< "Your roll is (" << first_die << ", "
<< sec_die << "}" << endl << endl;
/*
Get two new random values.
*/
first_die = rand() % (HIGH - LOW + 1) + LOW;
sec_die = rand() % (HIGH - LOW + 1) + LOW;
/*
Output second roll results.
*/
cout<< "My roll is (" << first_die << ", "
<< sec_die << "}" << endl << endl;
return 0;
}
```